

Detección de anomalías en servicio de TV-over-IP mediante autoencoder LSTM

Ing. Christopher Charaf

Carrera de Especialización en Inteligencia Artificial

Director: Esp. Ing. María Fabiana Cid (FIUBA)

Jurados:

A designar (pertenencia)

A designar (pertenencia)

A designar (pertenencia)

Ciudad Autónoma de Buenos Aires, junio de 2026

Resumen

En la presente memoria se describe el diseño y la implementación de un sistema de detección de anomalías para un servicio de televisión por protocolo de internet basado en interfaces de programación de aplicaciones, desarrollado para una empresa proveedora de plataformas de video sobre infraestructura en la nube. Para su concreción, fue necesaria la aplicación de aprendizaje profundo sobre series temporales mediante un autoencoder con redes neuronales recurrentes de memoria a largo plazo, junto con técnicas de ingeniería de características y de integración con sistemas de observabilidad de software como Prometheus, Grafana y Opsgenie.

Agradecimientos

Índice general

Resumen	I
1. Introducción general	1
1.1. Introducción general al tema	1
1.2. Contexto y motivación	2
1.3. Estado del arte	2
1.4. Objetivos del trabajo	4
2. Introducción específica	7
2.1. Frameworks y bibliotecas de IA/ML	7
2.2. Datasets y datos de entrenamiento	8
2.3. Modelos preentrenados y transferencia de aprendizaje	9
2.4. Infraestructura de cómputo	10
3. Diseño e implementación	13
3.1. Análisis del software	13
4. Ensayos y resultados	15
4.1. Pruebas funcionales del hardware	15
5. Conclusiones	17
5.1. Conclusiones generales	17
5.2. Próximos pasos	17
Bibliografía	19

Índice de figuras

1.1. Ejemplo conceptual de una métrica de servicio con un tramo de comportamiento anómalo.	2
1.2. Diagrama de flujo del sistema propuesto.	3

Índice de tablas

1.1. Comparación sintética de enfoques para detección de anomalías en métricas operativas (orientación conceptual).	4
2.1. Bibliotecas de terceros utilizadas en el sistema y su rol.	8
2.2. Descripción del conjunto de datos utilizado para entrenamiento y evaluación.	9
2.3. Entorno de cómputo utilizado para entrenamiento y despliegue. . .	10

Dedicatoria opcional.

Capítulo 1

Introducción general

En el presente capítulo se contextualiza la detección de anomalías en servicios de televisión por protocolo de internet (TV-over-IP) apoyados en interfaces de programación de aplicaciones (API), y se expone por qué conviene un modelo de aprendizaje profundo frente a umbrales fijos en el monitoreo de infraestructura. Se revisa el estado del arte de forma breve y se enuncian los objetivos del trabajo.

1.1. Introducción general al tema

Los servicios de video bajo demanda y de transmisión lineal sobre redes IP consolidaron en la última década un modelo en que la calidad percibida por el usuario depende tanto de la disponibilidad de los componentes de *backend* como del desempeño de la cadena completa: balanceadores, microservicios, bases de datos, colas de mensajes y sistemas de caché, entre otros. La operación de estos sistemas se apoya de manera casi universal en la recolección periódica de métricas técnicas (latencias, tasas de error, uso de CPU o memoria, etc.) que describen el estado del servicio en el tiempo.

El *machine learning* aporta métodos que aprenden, a partir de datos históricos, qué aspecto tiene el comportamiento nominal sin fijar a priori todas las reglas. En particular, las redes neuronales recurrentes con unidades de memoria a largo plazo (LSTM, por sus siglas en inglés) permiten modelar dependencias temporales en series multivariadas [1]. Los *autoencoders* basados en LSTM pueden entrenarse para reconstruir ventanas recientes de métricas. Cuando la reconstrucción falla de manera sistemática, el error sirve como señal de anomalía [2]. Este tipo de enfoque es comprensible para un lector familiarizado con monitoreo de infraestructura aunque no especializado en aprendizaje profundo: la idea central es que el modelo aprende un patrón habitual y marca desviaciones fuertes respecto de ese patrón.

Una anomalía en este contexto operativo puede entenderse como un patrón de comportamiento en esas señales que se aparta de lo observado cuando el sistema funciona de forma nominal, ya sea por degradación, por un incidente puntual o por una combinación de factores que los umbrales convencionales no caracterizan bien. La figura 1.1 ilustra esta idea sobre una métrica sintética observada a lo largo de un día: la banda verde representa el régimen nominal esperado para esa hora y la curva azul, las observaciones reales. El tramo rojo marca una ventana en la que el comportamiento se aparta del patrón habitual y representa el tipo de desvío sostenido que se busca identificar de manera automática. Detectar esas situaciones de manera temprana importa porque reduce el tiempo de detección de

incidentes y acota el impacto en la experiencia de visualización y en los acuerdos de nivel de servicio.

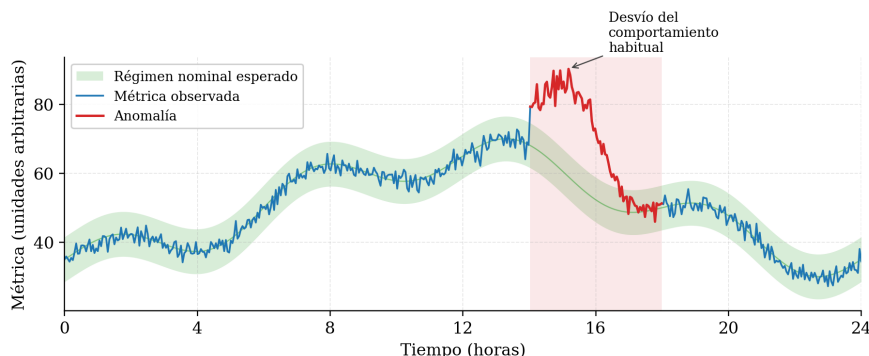


FIGURA 1.1. Ejemplo conceptual de una métrica de servicio con un tramo de comportamiento anómalo.

1.2. Contexto y motivación

La motivación del trabajo final surge de una aplicación real en el sector de plataformas de video empresarial: la organización de referencia provee servicios de TV-over-IP sobre infraestructura en la nube y requirió modernizar la forma en que se detectan fallos y degradaciones en producción. Las métricas del servicio se concentran en herramientas estándar de la industria, en particular en un sistema de acumulación de series temporales del tipo de Prometheus [3, 4], con muestreo en el orden de decenas de segundos.

Las prácticas tradicionales de alertas sobre umbrales fijos o reglas manuales siguen siendo útiles para síntomas claros, pero muestran debilidades frente a patrones multivariados, estacionalidades (por ejemplo variaciones por horario comercial o fines de semana) y dependencias que no se traducen en un único umbral intuitivo. Además, externalizar el análisis de métricas a terceros puede no ser deseable desde el punto de vista de privacidad y superficie de ataque.

Por ello se planteó un sistema que consume métricas ya disponibles a través de la API del monitoreo y construye ventanas temporales multivariadas. Sobre esas ventanas se aplica ingeniería de características acorde al dominio, por ejemplo codificación cíclica del tiempo y señales de contexto operativo.

Las ventanas resultantes alimentan un autoencoder LSTM que reconstruye las métricas, y el error de reconstrucción se compara contra un umbral. Cuando ese error supera el umbral, se dispara una alerta contextualizada hacia Opsgenie con un enlace al tablero correspondiente en Grafana, en caso contrario, el ciclo continúa con la siguiente ventana de métricas.

El flujo completo de la obtención de las métricas hasta la generación de la alerta, se ilustra en la figura 1.2.

1.3. Estado del arte

La detección de anomalías abarca desde métodos estadísticos y basados en distancia hasta técnicas de aprendizaje profundo adaptadas a datos secuenciales.

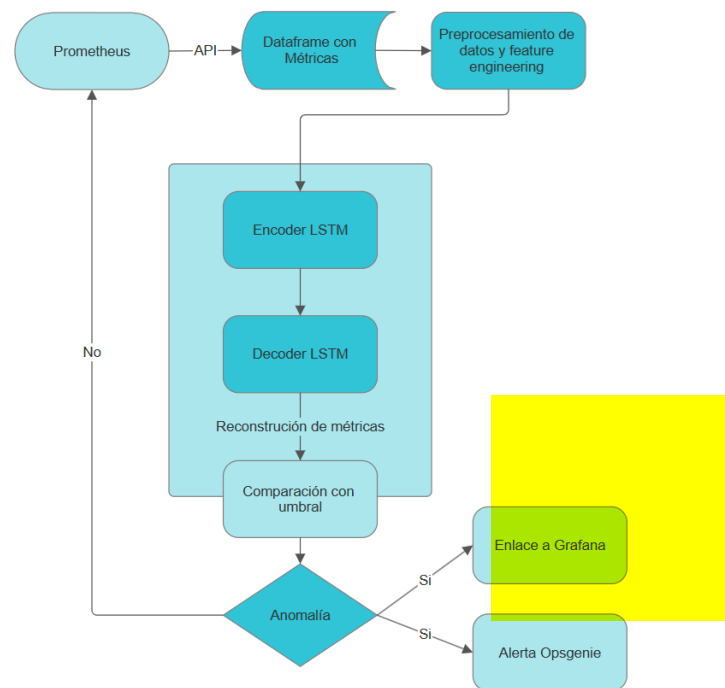


FIGURA 1.2. Diagrama de flujo del sistema propuesto.

Existe una vasta literatura de marcos de referencia y taxonomías [5]. En operaciones de software, los datos son en la práctica series multivariadas en las que distintas métricas capturan facetas acopladas del sistema, lo que motiva modelos que traten la ventana temporal completa en lugar de decidir variable por variable de forma independiente, para capturar no linealidades y dependencias largas.

Los métodos clásicos (controles por umbrales, suavizados exponenciales o *isolation forest* [6] sobre vectores de características) conservan ventajas de simplicidad e interpretabilidad, pero pueden exigir mucho ajuste manual cuando el número de series crece o cuando la normalidad cambia lentamente en el tiempo. Los autoencoders y las arquitecturas recurrentes ofrecen una capa adicional de expresividad para capturar no linealidades y dependencias largas.

La literatura ha mostrado resultados sólidos con codificadores–decodificadores LSTM entrenados para minimizar el error de reconstrucción en ventanas de entrenamiento consideradas normales [2]. Paralelamente, los ecosistemas de observabilidad comercial integran funciones de detección asistida o modelos propietarios. La propuesta de este trabajo se distingue por acoplarse al *stack* Prometheus–Grafana–Opsgenie [7, 8, 9] ya adoptado en la organización y por mantener la configuración, entrenamiento e inferencia bajo el control del operador, buscando un equilibrio entre sensibilidad y falsos positivos calibrable con el equipo de operaciones.

La tabla 1.1 resume de manera orientativa familias de enfoques y su relación típica con datos y métricas de evaluación. No pretende ser exhaustiva sino situar la línea elegida en el panorama habitual de ingeniería de confiabilidad y aprendizaje automático.

TABLA 1.1. Comparación sintética de enfoques para detección de anomalías en métricas operativas (orientación conceptual).

Enfoque	Tipo de modelo o regla	Datos típicos	Métricas de desempeño habituales
Umbrales fijos y reglas	Reglas declarativas, comparaciones con constantes	Series univariadas o pocas variables, supuestos estables	Tiempo de detección, acuerdo con incidentes conocidos, carga de falsas alertas.
Modelos clásicos de anomalía	Por ejemplo aislamiento, kNN, métodos estadísticos multivariados	Vectores de características por instante o ventana corta	Precisión, exhaustividad (<i>recall</i>), F1 si existe etiquetado parcial.
Modelos secuenciales profundos	Autoencoder LSTM, otras RNN, a veces combinadas con atención	Ventanas multivariadas de series temporales	Error de reconstrucción, AUC-PR, F1 sobre eventos etiquetados.
Productos comerciales de observabilidad	Algoritmos propietarios, a veces combinados con reglas	Telemetría agregada en plataforma	Definidas por el proveedor con un foco en integración.

1.4. Objetivos del trabajo

El objetivo general del trabajo consistió en diseñar e implementar un sistema inteligente capaz de detectar anomalías en tiempo de operación compatible con el monitoreo existente en un servicio de TV-over-IP basado en API, mediante un autoencoder LSTM entrenado sobre ventanas de métricas obtenidas desde Prometheus, con integración de alertas y enlaces de contexto con Opsgenie y Grafana, con miras a mejorar la capacidad de reacción frente a incidentes sin depender exclusivamente de umbrales estáticos ni de servicios de terceros para el núcleo analítico.

Los objetivos específicos, formulados de manera verificable, fueron los siguientes:

1. Identificar y preparar las métricas críticas del servicio disponibles vía API de Prometheus, incluyendo normalización, variables de contexto temporal (por ejemplo codificación cíclica de la hora) y construcción de ventanas deslizando acordes al periodo de muestreo del entorno.
2. Entrenar y ajustar un modelo autoencoder LSTM capaz de reconstruir el comportamiento nominal del sistema y traducir el error de reconstrucción en una señal de anomalía gobernada por un umbral calibrado con el equipo de operaciones.
3. Integrar el flujo de inferencia con mecanismos de alerta y la generación de enlaces a paneles en Grafana que permitan inspeccionar el intervalo temporal asociado a cada anomalía detectada.
4. Evaluar el sistema sobre datos históricos representativos del régimen normal de operación, reportando al menos la tasa de falsos positivos y la capacidad de detección en escenarios acordados con las partes interesadas, de

forma que los resultados en esta memoria permitan contrastar el desempeño frente a líneas base definidas en el capítulo de ensayos.

Capítulo 2

Introducción específica

En el presente capítulo se describen los recursos de software (librerías, bibliotecas, conjuntos de datos e infraestructura de cómputo de terceros) que soportan el sistema desarrollado como parte de esta memoria técnica. **Bajo estas premisas, se presenta el contexto técnico bajo el cual el modelo fue entrenado, evaluado y desplegado.**

2.1. Frameworks y bibliotecas de IA/ML

La implementación del autoencoder LSTM se apoyó en **TensorFlow** y su interfaz **Keras** [10, 11], que ofrecen una capa de alto nivel para definir, entrenar y serializar redes neuronales profundas. La elección obedeció a tres criterios: la disponibilidad nativa de capas recurrentes (LSTM, RepeatVector, TimeDistributed) que permiten expresar la arquitectura codificador-decodificador con pocas líneas. La integración estable con el ecosistema Python usado para procesamiento de datos y la portabilidad del modelo entrenado a contenedores en CPU, sin requerir aceleradores específicos para la inferencia.

Para el preprocesamiento se utilizó **scikit-learn** [12], en particular su **MinMaxScaler** configurado con cotas fijas. Este criterio busca desacoplar la normalización de la distribución particular de los datos de entrenamiento. La persistencia del preprocesador, necesaria para garantizar la misma transformación entre entrenamiento e inferencia, se gestionó con **pickle**, una biblioteca de serialización binaria habitual en el ecosistema de **scikit-learn**.

La manipulación de las series temporales se apoyó en **NumPy** [13] y **pandas** [14], estándares en Python para arreglos n-dimensionales y tablas indexadas por tiempo. La obtención de métricas desde Prometheus se realizó vía HTTP con la biblioteca **requests**, mientras que la instrumentación del servicio simulado utilizó el cliente oficial **prometheus_client**, que expone contadores e histogramas en el formato esperado por el recolector. La carga de archivos de configuración en formato YAML se delegó en **PyYAML** y la generación de gráficos de evaluación *offline* se realizó con **matplotlib** y **boorn**. La tabla 2.1 resume las bibliotecas principales y el rol que cumplen dentro del sistema.

La decisión de adoptar **TensorFlow** frente a alternativas como **PyTorch** se fundamentó en la compatibilidad con el flujo de despliegue de la organización (contenedores ligeros sobre Python en CPU) y en la estabilidad del formato de pesos **.weights.h5**, que se acompaña de un archivo de configuración en JSON con la arquitectura para reconstruir el modelo en inferencia sin acoplarse a una versión particular del entorno de entrenamiento.

TABLA 2.1. Bibliotecas de terceros utilizadas en el sistema y su rol.

Biblioteca	Versión	Rol en el sistema
TensorFlow / Keras	2.8	Definición y entrenamiento del autoencoder LSTM.
scikit-learn	1.0	Normalización (<code>MinMaxScaler</code>) con cotas fijas.
joblib	1.1	Persistencia del preprocesador entrenado.
NumPy	1.19	Operaciones con arreglos n-dimensionales y cálculo del error de reconstrucción.
pandas	1.3	Manipulación de series temporales y construcción de ventanas.
requests	2.26	Consultas HTTP a la API de Prometheus.
prometheus_client	0.17	Exposición de métricas en el servicio simulado.
PyYAML	5.4	Carga de archivos de configuración.
matplotlib / seaborn	3.4 / 0.11	Gráficos de evaluación offline del modelo.

2.2. Datasets y datos de entrenamiento

El conjunto de datos utilizado para entrenar y evaluar el sistema corresponde a telemetría operativa del servicio de TV-over-IP de la empresa cliente, recolectada con un intervalo de muestreo de 30 segundos sobre una ventana histórica de 90 días. La fuente primaria es la instancia interna de Prometheus de la organización [3, 4], consultada a través de su API HTTP de rango (`api/v1/query_range`) con las consultas PromQL definidas en el archivo de configuración `config/data.yaml` del repositorio. Al tratarse de telemetría propia de un servicio en producción, no se utilizaron *datasets* públicos: el dominio (calidad de servicio en *streaming* sobre infraestructura específica) no encuentra equivalencia directa en repositorios abiertos como *Numenta NAB* o *Yahoo Webscope*, y la representatividad del régimen nominal exigía datos del propio entorno.

Para los entornos de desarrollo y evaluación reproducible se incorporó un servicio simulado (*mock service*) basado en *Flask*, instrumentado con la misma familia de métricas que el servicio real. Este simulador comparte un núcleo de simulación con un generador sintético offline (`traffic_simulation_core.py`), de modo que las series obtenidas por consulta a Prometheus y las series sintéticas producidas para reproducibilidad atraviesan las mismas reglas de agregación que las consultas PromQL del sistema productivo. Con este criterio se evita la divergencia entre los datos de entrenamiento y los de inferencia.

Cada observación temporal del conjunto de datos consta de cinco métricas técnicas del servicio: tasa de peticiones por segundo, percentil 95 de latencia, memoria residente del proceso, tasa de errores y uso de CPU. Sobre estas métricas se aplicó ingeniería de variables temporales, en particular codificación cíclica de la hora del día y del día de la semana mediante senos y cosenos, junto con indicadores binarios de fin de semana y horario nocturno. La tabla 2.2 sintetiza las características del conjunto resultante antes y después de la transformación a ventanas deslizantes.

TABLA 2.2. Descripción del conjunto de datos utilizado para entrenamiento y evaluación.

Atributo	Valor o descripción
Origen de datos	Prometheus (instancia interna de la empresa cliente) con mock service y generador sintético de núcleo compartido para desarrollo.
Período cubierto	90 días corridos previos al entrenamiento.
Intervalo de muestreo	30 segundos.
Cantidad aproximada de muestras	259 200 puntos por métrica.
Métricas técnicas	request_rate, latency_p95, memory_usage, error_rate y cpu_usage.
Variables temporales derivadas	hour_sin, hour_cos, dayofweek_sin, dayofweek_cos, is_weekend e is_night.
Dimensión de cada ventana	20 pasos $\times$ 11 variables.
Longitud temporal de la ventana	10 minutos.
Particionado entrenamiento / validación	80 % / 20 % en orden temporal (último tramo como validación).
Etiquetas	Régimen no supervisado: el modelo se entrenó sobre tramos considerados nominales.

Cabe aclarar que el problema se planteó como detección no supervisada: el conjunto de entrenamiento se compone de tramos en los que el servicio operó dentro de su régimen nominal y no se dispone de un etiquetado fiable de anomalías históricas a la escala del dataset completo. La calibración del umbral y la evaluación cualitativa frente a escenarios anómalos se desarrollarán mas adelante.

2.3. Modelos preentrenados y transferencia de aprendizaje

El sistema no incorpora modelos preentrenados ni esquemas de *transfer learning*. La razón es **doble**: por un lado, no existen modelos públicamente disponibles entrenados sobre telemetría operativa de servicios de TV-over-IP con la misma combinación de métricas y régimen de muestreo, por lo que la transferencia desde dominios distintos (texto, visión o series financieras) no resultaba directamente aplicable. Por otro lado, el tamaño del modelo propuesto (codificador con tres capas LSTM de 64, 32 y 16 unidades, decodificador simétrico) y el volumen disponible de datos del régimen nominal son adecuados para un entrenamiento desde cero en tiempos razonables sobre hardware modesto, sin necesidad de inicializaciones de gran capacidad. Por estos motivos se optó por entrenar el modelo íntegramente sobre datos del dominio, criterio que también facilita la actualización periódica del modelo a medida que el régimen de operación evoluciona con sucesivas iteraciones del producto.

2.4. Infraestructura de cómputo

El sistema fue concebido para ejecutarse en infraestructura propia, sin envío de telemetría a servicios externos. Tanto el entrenamiento como la inferencia se realizan en **CPU**; el tamaño reducido del modelo y de la ventana de inferencia hacen innecesario el uso de aceleradores gráficos. El entorno de desarrollo se orquestó con Docker y Docker Compose [15], lo que permite **levantar** de forma reproducible los servicios involucrados desde un único archivo de declaración.

El detector se distribuye como una imagen basada en `python:3.10-slim`, con un usuario sin privilegios y montajes de los directorios de modelos, configuración y registros para preservar artefactos entre reinicios. La asignación de recursos definida en `docker-compose.yml` contempla un límite de una CPU virtual y 2 GB de memoria RAM para el contenedor de detección, lo que resulta suficiente para una inferencia cada 30 segundos sobre ventanas de 10 minutos. Los servicios auxiliares (servicio simulado, Prometheus y Grafana [7, 8]) se aíslan en una red puente dedicada y se exponen solo los puertos necesarios para inspección manual durante el desarrollo.

El entrenamiento se ejecutó en un equipo de desarrollo con procesador de uso general y sin GPU dedicada. Con una arquitectura de tres capas LSTM por rama, un tamaño de lote (*batch*) de 256 muestras y un máximo de 30 épocas con parada temprana, el ciclo completo se completó en un tiempo del orden de minutos a decenas de minutos, según la cantidad de ventanas resultantes del período de 90 días. Esta capacidad de reentrenar en un horizonte corto fue un criterio explícito de diseño, dado que el régimen de operación del servicio puede evolucionar con cambios de versión del producto o con variaciones estacionales del tráfico.

La tabla 2.3 resume el entorno técnico bajo el que se entrenó y desplegó el sistema.

TABLA 2.3. Entorno de cómputo utilizado para entrenamiento y despliegue.

Componente	Descripción
Lenguaje y entorno base	Python 3.10 sobre imagen <code>python:3.10-slim</code> .
Orquestación de servicios	Docker Compose con perfil <code>dev</code> para entorno completo de pruebas.
Recursos del contenedor de detección	1.0 CPU virtual y 2 GB de memoria RAM (límite declarado en <code>docker-compose.yml</code>).
Aceleración por GPU	No utilizada ; todo el cómputo se realizó en CPU.
Servicios auxiliares	Prometheus (almacenamiento de series temporales), Grafana (visualización) y servicio simulado en Flask para pruebas.
Red	Red puente <code>monitoring</code> en Docker, sin exposición pública.
Destino de despliegue productivo	Infraestructura propia de la empresa cliente sobre AWS, sin dependencia de servicios analíticos de terceros.

La integración con Opsgenie [9] para la emisión de alertas y la generación de enlaces contextuales a Grafana se realizó mediante sus APIs públicas a través de HTTPS, sin agentes adicionales en el contenedor. De este modo, la infraestructura

agregada por el sistema se reduce a un único servicio adicional dentro de la red interna, lo que respeta los requerimientos de seguridad y privacidad establecidos para el proyecto.

Capítulo 3

Diseño e implementación

Todos los capítulos deben comenzar con un breve párrafo introductorio que indique cuál es el contenido que se encontrará al leerlo. La redacción sobre el contenido de la memoria debe hacerse en presente y todo lo referido al proyecto en pasado, siempre de modo impersonal.

3.1. Análisis del software

La idea de esta sección es resaltar los problemas encontrados, los criterios utilizados y la justificación de las decisiones que se hayan tomado.

Se puede agregar código o pseudocódigo dentro de un entorno `lstlisting` con el siguiente código:

```
\begin{lstlisting}[caption= "un epígrafe descriptivo"]
  las líneas de código irían aquí...
\end{lstlisting}
```

A modo de ejemplo, se muestra el fragmento de código [3.1](#):

```
1 #define MAX_SENSOR_NUMBER 3
2 #define MAX_ALARM_NUMBER 6
3 #define MAX_ACTUATOR_NUMBER 6
4
5 uint32_t sensorValue[MAX_SENSOR_NUMBER];
6 FunctionalState alarmControl[MAX_ALARM_NUMBER]; //ENABLE or DISABLE
7 state_t alarmState[MAX_ALARM_NUMBER]; //ON or OFF
8 state_t actuatorState[MAX_ACTUATOR_NUMBER]; //ON or OFF
9
10 void vControl() {
11
12     initGlobalVariables();
13
14     period = 500 ms;
15
16     while(1) {
17
18         ticks = xTaskGetTickCount();
19
20         updateSensors();
21
22         updateAlarms();
23
24         controlActuators();
25
26         vTaskDelayUntil(&ticks, period);
27     }
```

28 }

CÓDIGO 3.1. Pseudocódigo del lazo principal de control.

Capítulo 4

Ensayos y resultados

Todos los capítulos deben comenzar con un breve párrafo introductorio que indique cuál es el contenido que se encontrará al leerlo. La redacción sobre el contenido de la memoria debe hacerse en presente y todo lo referido al proyecto en pasado, siempre de modo impersonal.

4.1. Pruebas funcionales del hardware

La idea de esta sección es explicar cómo se hicieron los ensayos, qué resultados se obtuvieron y analizarlos.

Capítulo 5

Conclusiones

Todos los capítulos deben comenzar con un breve párrafo introductorio que indique cuál es el contenido que se encontrará al leerlo. La redacción sobre el contenido de la memoria debe hacerse en presente y todo lo referido al proyecto en pasado, siempre de modo impersonal.

5.1. Conclusiones generales

La idea de esta sección es resaltar cuáles son los principales aportes del trabajo realizado y cómo se podría continuar. Debe ser especialmente breve y concisa. Es buena idea usar un listado para enumerar los logros obtenidos.

En esta sección no se deben incluir ni tablas ni gráficos.

Algunas preguntas que pueden servir para completar este capítulo:

- ¿Cuál es el grado de cumplimiento de los requerimientos?
- ¿Cuán fielmente se pudo seguir la planificación original (cronograma incluido)?
- ¿Se manifestó algunos de los riesgos identificados en la planificación? ¿Fue efectivo el plan de mitigación? ¿Se debió aplicar alguna otra acción no contemplada previamente?
- Si se debieron hacer modificaciones a lo planificado ¿Cuáles fueron las causas y los efectos?
- ¿Qué técnicas resultaron útiles para el desarrollo del proyecto y cuáles no tanto?

5.2. Próximos pasos

Acá se indica cómo se podría continuar el trabajo más adelante.

Bibliografía

- [1] Sepp Hochreiter y Jürgen Schmidhuber. «Long Short-Term Memory». En: *Neural Computation* 9.8 (1997), págs. 1735-1780.
- [2] Pankaj Malhotra et al. «Long Short Term Memory Networks for Anomaly Detection in Time Series». En: *Proceedings of ESANN*. 2015.
- [3] Brian Brazil. *Prometheus: Up & Running. Infrastructure and Application Performance Monitoring*. Sebastopol, CA: O'Reilly Media, 2018.
- [4] Prometheus Authors. *Prometheus – Monitoring system and time series database*. <https://prometheus.io/>. Visitado el 2026-05-09. 2024.
- [5] Varun Chandola, Arindam Banerjee y Vipin Kumar. «Anomaly Detection: A Survey». En: *ACM Computing Surveys* 41.3 (2009), 58:1-58:58.
- [6] Fei Tony Liu, Kai Ming Ting y Zhi-Hua Zhou. «Isolation Forest». En: *Proceedings of the 2008 Eighth IEEE International Conference on Data Mining (ICDM)*. Pisa, Italy: IEEE Computer Society, 2008, págs. 413-422. DOI: [10.1109/ICDM.2008.17](https://doi.org/10.1109/ICDM.2008.17).
- [7] Eric Salituro. *Learn Grafana 7.0: A beginner's guide to getting well versed in analytics, interactive dashboards, and monitoring*. Birmingham: Packt Publishing, 2020.
- [8] Grafana Labs. *Grafana: The open observability platform*. <https://grafana.com/>. Visitado el 2026-05-09. 2024.
- [9] Atlassian. *Opsgenie: Modern incident management and response*. <https://www.atlassian.com/software/opsgenie>. Visitado el 2026-05-09. 2024.
- [10] Martín Abadi et al. «TensorFlow: A system for large-scale machine learning». En: *Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. 2016, págs. 265-283.
- [11] François Chollet et al. *Keras*. <https://keras.io>. Visitado el 2026-05-11. 2015.
- [12] F. Pedregosa et al. «Scikit-learn: Machine Learning in Python». En: *Journal of Machine Learning Research* 12 (2011), págs. 2825-2830.
- [13] Charles R. Harris et al. «Array programming with NumPy». En: *Nature* 585.7825 (2020), págs. 357-362. DOI: [10.1038/s41586-020-2649-2](https://doi.org/10.1038/s41586-020-2649-2).
- [14] Wes McKinney. «Data Structures for Statistical Computing in Python». En: *Proceedings of the 9th Python in Science Conference*. 2010, págs. 56-61.
- [15] Dirk Merkel. «Docker: lightweight Linux containers for consistent development and deployment». En: *Linux Journal* 2014.239 (2014).